



TXSDK 开发入门指南



珠海泰芯半导体有限公司
Zhuhai Taixin Semiconductor Co., Limited

珠海市高新区港湾一号科创园港
11 栋 3 楼

版权所有 侵权必究

Copyright © 2023 by TaiXin All Rights Reserved

保密等级	A	TXSDK 开发入门指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

责任与版权

责任限制

由于产品版本升级或者其他原因，本文档会不定期更新。除非另行约定，泰芯半导体有限公司对本文档所有内容不提供任何担保或授权。

客户应在遵守法律、法规和安全要求的前提下进行产品设计，并做充分验证。泰芯半导体有限公司对应用帮助或客户产品设计不承担任何义务。客户应对其使用泰芯半导体有限公司的产品和应用自行负责。

在适用法律允许的范围内，泰芯半导体有限公司在任何情况下，都不对因使用本文档相关内容及本文档描述的产品而产生的损失和损害进行超过购买支付价款的赔偿（除在涉及人身伤害的情况中根据适用的法律规定的损害赔偿外）。

版权申明

泰芯半导体有限公司保留随时修改本文档中任何信息的权利，无需提前通知且不承担任何责任。

未经泰芯半导体有限公司书面同意，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。除非获得相关权利人的许可，否则，任何人不能以任何形式对前述软件进行复制、分发、修改、摘录、反编译、反汇编、解密、反向工程、出租、转让、分许可等侵犯本文档描述的享有版权的软件版权的行为，但是适用法禁止此类限制的除外。



珠海泰芯半导体有限公司
Zhuhai TaiXin Semiconductor Co., Limited

珠海市高新区港湾一号科创园港
11 栋 3 楼

版权所有 侵权必究

Copyright © 2023 by TaiXin All Rights Reserved

保密等级	A	TXSDK 开发入门指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

修订记录

日期	版本	描 述	修订人
2024-05-07	V1.0		TX

保密等级	A	TXSDK 开发入门指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

目录

TXSDK 开发入门指南 1

1. 概述1

 1.1. SDK 版本信息 1

2. SDK 目录结构2

3. SDK 启动流程3

 3.1. startup.S 3

 3.2. system.c 3

 3.3. main.c 5

 3.4. 二次 loader 启动流程 5

4. SDK 配置头文件6

 4.1. sys_config.h 6

 4.2. project_config.h 7

5. SDK 内存管理8

 5.1. SRAM 内存 8

 5.2. PSRAM 内存 10

 5.3. 内存分配 11

 5.4. 系统 heap 11

 5.5. 自定义 heap 12

6. SDK 开发基本规范 14

 6.1. 头文件引用 14

 6.2. os_xx API 说明 15

7. SDK 设备驱动 16

 7.1. SDK 设备管理 16

保密等级	A	TXSDK 开发入门指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

7.2. 驱动抽象层 17

8. SDK OS 抽象层 18

8.1. OS Task 18

8.2. OS Mutex 19

8.3. OS Semaphore 20

8.4. OS MsgQueue 20

8.5. OS Timer 21

8.6. OS Event 22

8.7. OS Condv 23

8.8. OS WorkQueue/Work 23

9. SDK 事件消息 25

9.1. 事件类型 25

9.2. 发布事件 26

9.3. 订阅事件 27

10. SDK POSIX 接口 28

10.1. FS 接口 28

10.2. Socket 接口 28

10.3. Pthread 接口 28

11. SDK 参数管理 29

11.1. API 列表 29

11.2. 参数分区 30

11.3. 参数扩展 31

11.4. 自定义参数区 31

12. SDK C++代码支持 32

保密等级	A	TXSDK 开发入门指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

13. SDK 内置小模块 33

 13.1. ringbuffer 33

 13.2. 字符串转换 35

 13.3. print 输出 35

 13.4. errlog 功能 37

14. SDK 固件生成 39

 14.1. 固件生成过程 39

 14.2. 固件烧录 39

15. SDK 调试辅助 40

 15.1. 串口固件升级 40

 15.2. CPU 使用率 41

 15.3. 内存分配信息 42

 15.4. Task 状态分析 43

1. 概述

TXSDK 是泰芯半导体发布的 WiFi/音视频系列芯片开发 SDK。本文档是 TXSDK 的开发入门指南，介绍了 SDK 的代码框架，基本开发规范以及 SDK 的基础功能模块。

本文档适用于泰芯所有的 WiFi/音视频芯片 SDK，作为基础文档，而各个芯片的特有功能部分请查阅芯片的专用文档。

1.1. SDK 版本信息

SDK 支持开发多种类型的方案，例如低功耗 WiFi 模组，无线网桥，IoT 模组，无线图传等方案。在发布的 SDK 版本信息中包含了 SDK 版本，SDK 方案类型信息。

例如：TXW81x_IOT-v2.5.3.6-28970

其中：

- TXW81x：芯片系列名称：TXW81x 系列
- IOT：SDK 适用的方案类型：IoT 类方案
- v2.5.3.6：SDK 版本号：主版本-2，分支版本-5，Patch 版本-3，方案类型-6（对应 IoT）
- 28970：内部版本号，该版本号是递增的唯一值。

2. SDK 目录结构

SDK 解压后的目录结构如下所示：

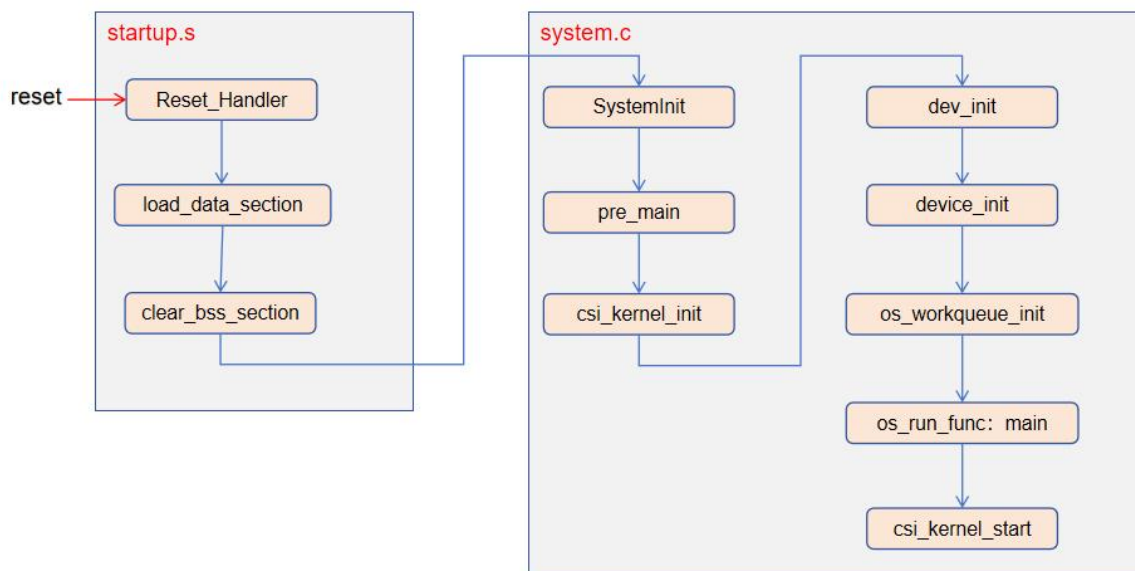


目录说明：

- **csky**: SDK 使用的 RTOS 内核代码。csky 目录使用的是 alios 内核。
- **doc**: SDK 开发文档目录
- **libs**: SDK 发布的库文件
- **project**: SDK 编译工程目录
- **sdk**: SDK 代码目录
 - **sdk/chip**: 芯片启动代码目录（示例是 TXW81x 系列芯片的启动代码）
 - **sdk/driver**: 芯片驱动代码目录
 - **sdk/hal**: SDK 驱动抽象层代码目录
 - **sdk/include**: SDK 头文件目录
 - **sdk/lib**: SDK library 代码目录。SDK 默认支持了 lwip 协议栈，libcurl，posix 适配 API，mbedtls 加密库，mmc 协议栈，USB 协议栈等。

3. SDK 启动流程

sdk/chip 目录是芯片启动代码目录，芯片启动流程相关的代码文件是 startup.s 和 system.c。启动流程如下图所示：



3.1. startup.S

startup.S 文件是芯片启动汇编代码文件。各个芯片的 startup.S 文件的代码实现思路基本保持一致，所实现的功能包括：中断向量表定义，复位入口函数，加载数据段，清零 BSS 段，然后跳转执行 Systeminit 函数和 pre_main 函数。

3.2. system.c

system.c 文件是芯片系统初始化代码文件。各个芯片的 system.c 文件的代码实现基本保持一致。该文件主要实现了 2 个函数：Systeminit 函数和 pre_main 函数。

Systemint 函数实现了对芯片的初始化设置，包括：CPU Cache 设置，中断向量表设置，中断初始化，CPU 时钟频率设置，CPU TICK 设置。

pre_main 函数实现了软件系统的初始化，包括 Heap 初始化，RTOS 内核初始化，SDK

设备管理模块初始化，外设驱动模块初始化，主工作队列初始化，调用 main 函数，启动 RTOS 内核。

```

_init void pre_main(void)
{
    sdk_version = SDK_VERSION;
    svn_version = SVN_VERSION;
    app_version = APP_VERSION;
    assert_holdup = ASSERT_HOLDUP;
    save_boot_loader_addr();
    if(NOW_PSRAM > PSRAM_DEF(E_PSRAM)) {
        uint8_t res = psram_init(APS1604M_3SQR, (60*1000000),512);
        set_psram_status(res);
    }
    #if MPOOL_ALLOC
        malloc_init();
        malloc_psram_init();
    #endif
    #ifdef CSKY_OS
        csi_kernel_init();
    #endif
    #ifdef OHOS
        LOS_KernelInit();
    #endif
    dev_init();
    device_init();
    #ifdef CONFIG_SLEEP
        sys_sleepcb_init();
    #endif
    #ifdef CONFIG_SLEEP
        sys_sleepdata_init();
    #endif
    sysctrl_rst_lmac_phy();
    sysctrl_efuse_validity_handle();
    VERSION_SHOW();
    module_version_show();
    sys_reset_show();
    os_workqueue_init(&main_wkq, "MAIN", OS_TASK_PRIORITY_NORMAL, 2048);
    mainwkq_monitor_init();
    os_run_func((os_run_func_t)main, 0, 0, 0);
    #ifdef CSKY_OS
        csi_kernel_start();
    #endif
    #ifdef OHOS
        LOS_Start();
    #endif
} « end pre_main »

```

system heap initialization

RTOS kernel initialization

system device manager initialization

system device driver initialization

low power module initialization

main workqueue initialization

run main function

start RTOS kernel

```

void device_init(void)
{
    int32 flash_size = sysctrl_get_chip_flash_size();
    if (flash_size > 0) {
        flash0.size = flash_size;
    }

    hggpio_v4_attach(HG_GPIOA_DEVID, &gpioa);
    hggpio_v4_attach(HG_GPIOB_DEVID, &gpiob);
    hggpio_v4_attach(HG_GPIOC_DEVID, &gpioc);
    hggpio_v4_attach(HG_GPIOD_DEVID, &gpiod);
    hgadc_v0_attach(HG_ADC0_DEVID, &adc);
    hguart_v2_attach(HG_UART0_DEVID, &uart0);
    hguart_v4_attach(HG_UART4_DEVID, &uart4);
    hg_m2m_dma_dev_attach(HG_M2MDMA_DEVID, &mem_dma);
    hgtimer_v4_attach(HG_TIMER0_DEVID, &timer0);
    hg_crc_attach(HG_CRC_DEVID, &crc32_module);
    hg_sysaes_v3_attach(HG_HWAES0_DEVID, &sysaes);
    hgspi_v3_attach(HG_SPI0_DEVID, &spi0);
    hgspi_xip_attach(HG_SPI7_DEVID, &spi7);
    spi_nor_attach(&flash0, HG_FLASH0_DEVID);
    hgi2c_v1_attach(HG_I2C1_DEVID, &iic1);
    hgi2c_v1_attach(HG_I2C2_DEVID, &iic2);
    uart_open((struct uart_device *)&uart0, 115200);
    m2mdma = (struct dma_device *)&mem_dma;
    console_handle = &uart0;
} « end device_init »

```

system driver initialization

3.3. main.c

`main.c` 是 SDK 启动流程的最后阶段，用于进行应用代码初始化。不同类型方案的 `main.c` 差异可能会比较大，但是基础的初始化基本一致，包括：参数区加载，SDK 事件模块初始化，AT 指令模块初始化，WiFi 功能初始化，网络功能（lwip）初始化。

如前面所示 `main` 函数是由 `os_run_func` 函数启动执行的。不同于常见的 `main` 函数，SDK 的 `main` 函数是在主工作队列中执行，main 函数执行后会返回，没有 while 循环。所以在 `main` 函数中添加初始化代码时，不能添加 `while` 循环。SDK 的工作队列机制在后续的章节会详细介绍。

3.4. 二次 loader 启动流程

为了应对更加的复杂的应用方案，SDK 支持二次 loader 启动。

TBD...

4. SDK 配置头文件

SDK 有 2 个基础配置文件，分别是 `sys_config.h` 和 `project_config.h`。在这 2 个头文件中定义了很多宏定义开关，用于控制 SDK 功能的编译。

sys_config.h: 定义了 SDK 所需宏定义的默认值，通常无需修改此文件。

project_config.h: 在此文件中重定义了需要修改的宏定义。

4.1. sys_config.h

`sys_config.h` 文件定义了 SDK 所需宏定义的默认值，在该文件的起始位置引用了 `project_config.h`，优先使用 `project_config.h` 定义的宏定义值。

```
#ifndef __SDK_SYS_CONFIG_H__
#define __SDK_SYS_CONFIG_H__

#include "project_config.h"

#define PROJECT_TYPE PRO_TYPE_IOT

#define ATCMD_LINK_MAX (8)

#ifndef SYSCFG_ENABLE
#define SYSCFG_ENABLE 1
#endif

#ifndef MPOOL_ALLOC
#define MPOOL_ALLOC 1
#endif
```

`sys_config.h` 中几个重要的宏定义说明：

- **SYS_HEAP_SIZE**：定义 SDK 的 heap 大小。请根据实际方案的 heap 需求，调整这个宏定义值。所有的 `malloc` 操作都会从 heap 中进行分配。
- **WIFI_RX_BUFF_SIZE**：定义 WiFi 功能的 rx buffer 大小，影响 WiFi 的 RX 性能。根据实际方案对 WiFi 接收性能的需求，可以适当调整这个值。
- **SKB_POOL_SIZE**：WiFi 功能的 tx buffer 大小，影响 WiFi TX 性能。这个宏定义值无需修改，宏定义会根据 `SYS_HEAP_SIZE` 和 `WIFI_RX_BUFF_SIZE` 的值进行自动

计算。所以修改 SYS_HEAP_SIZE 和 WIFI_RX_BUFF_SIZE 的值会影响 WiFi 功能的 tx buffer 大小。

4.2. project_config.h

project_config.h 用于对宏定义进行重定义，不需要去修改 sys_config.h。

当存在多个类似方案，仅仅是部分宏定义值不一样时，可以在 project_config.h 中可以添加多个 PROJECT_ID 进行选择编译。

```
1  #ifndef __SYS_PROJECT_CONFIG_H__
2  #define __SYS_PROJECT_CONFIG_H__
3
4  #define PROJECT_ID 2
5  /*
6   * PROJECT_ID :
7   * 2 IOT_AT
8   */
9
10 #if (PROJECT_ID == 2)
11     #define PIN_UART0_TX      PC_3
12     #define PIN_UART0_RX      PC_2
13     #define ATCMD_UARTDEV     HG_UART0_DEVID
14     #define SYS_IOT_ATCMD      1
15     #define SYS_APP_DHCPD      0
16     #define LWIP_RAW           1
17     #define DCDC_EN           1
18
19     //启动外设总开关demo的宏
20     // #define EXAMPLES
21
```

5. SDK 内存管理

SDK 的内存管理是在编译阶段静态划分的, 整个 SRAM 被 data/bss 段占用后剩余的内存被分成了 3 部分: **HEAP**, **WiFi RX buffer** 和 **WiFi TX Buffer**。在 `sys_config.h` 中由 3 个宏定义决定各自的大小: `SYS_HEAP_SIZE`, `WIFI_RX_BUFF_SIZE` 和 `SKB_POOL_SIZE`, 这 3 部分内存划分的说明在 4.1 章节。

部分型号芯片有内置 PSRAM 或外挂 PSRAM, 可以根据 PSRAM 的用途进行 PSRAM 进行管理。可以将 PSRAM 作为 HEAP 使用, 也可以将其作为特定的 buffer 使用。

5.1. SRAM 内存

不同型号芯片的内置 SRAM 容量会不一样, 在工程的 `ld` 文件中定义了程序可用的 SRAM 区间。在 `ld` 文件中定义了 2 个变量: `__heap_end` 和 `__heap_start`。这 2 个变量就是 data/bss 段占用后的剩余 sram 区间, 该区间又会分成 3 部分: **HEAP**, **WiFi RX buffer** 和 **WiFi TX Buffer**。编译完成后也可以从 `map` 文件中得知 `__heap_start` 的大小。

```

    .sram_heap : {
        . = ALIGN(0x4) ;
        heap_start = .;
    } > REGION_BSS

    .psram_heap : {
        . = ALIGN(0x4) ;
        __psram_heap_start = .;
    } > REGION_PSRAM_HEAP

```

```

23 MEMORY
24 {
25     I-SRAM : ORIGIN = 0x0          , LENGTH = 0x80000 /* I-SRAM 128KB */
26     D-SRAM : ORIGIN = 0x20000000 , LENGTH = 0x80000 /* D-SRAM 512KB */
27     O-SRAM : ORIGIN = 0x50000000 , LENGTH = 0x80000 /* off-chip SRAM 8MB */
28     SRAM   : ORIGIN = 0x20000000 , LENGTH = 0x43C00 /* on-chip SRAM 288KB */
29     SRAM2  : ORIGIN = 0x20051000 , LENGTH = 0x1800
30     FLASH : ORIGIN = 0x18000000 , LENGTH = 0x200000
31     PSRAM  : ORIGIN = 0x38000000 , LENGTH = 0x200000
32 }
33
34
35 PROVIDE (__ram_end = 0x20043C00);
36 PROVIDE (__heap_end = 0x20043C00);
37 PROVIDE (__psram_heap_end = 0x38200000);
38

```

```

#ifndef HEAP_TAIL_ROOM
#define HEAP_TAIL_ROOM 0
#endif

#define SRAM_POOL_START (srampool_start)
#define SRAM_POOL_SIZE (srampool_end - srampool_start - HEAP_TAIL_ROOM)

#ifndef SYS_HEAP_SIZE
#define SYS_HEAP_SIZE (60*1024)
#endif

#ifndef WIFI_RX_BUFF_SIZE
#define WIFI_RX_BUFF_SIZE (36*1024)//(17*1024)
#endif

#define SYS_HEAP_START (SRAM_POOL_START)
#define WIFI_RX_BUFF_ADDR (SYS_HEAP_START+SYS_HEAP_SIZE)
#define SKB_POOL_ADDR (WIFI_RX_BUFF_ADDR+WIFI_RX_BUFF_SIZE)
#define SKB_POOL_SIZE (SRAM_POOL_START+SRAM_POOL_SIZE-SKB_POOL_ADDR)

```

```

__SYS_INIT void SystemInit(void)
{
    srampool_start = (uint32)&__heap_start;
    srampool_end = (uint32)&__heap_end;
    #if defined(PSRAM_HEAP)
        psrampool_start = (uint32)&__psram_heap_start;
        psrampool_end = (uint32)&__psram_heap_end;
    #endif
    #if SYS_CACHE_ENABLE
        cache_open();
    #endif
}

```

defined in ld script file

如上图所示，SRAM_POOL_SIZE 是 sram 剩余总空间（在 SystemInit 时根据 ld 文件变量 __heap_start 和 __heap_end 计算）。这部分空间又被分成了 SYS_HEAP, WIFI_RX_BUFFER, WIFI_TX_BUFFER 3 部分。

HEAP_TAIL_ROOM 是在 sram 的尾巴上预留一部分空间。可以根据方案需求，预留部分 memory 用作特殊用途。

5.2. PSRAM 内存

芯片的内置 PSRAM 或外挂 PSRAM 可以作为 HEAP 使用,也可以作为特定的 buffer 使用,也可以用来运行代码。不同型号的芯片, PSRAM 的使用可能会有不同的规则或限制条件,具体情况需要查阅芯片资料或咨询 FAE。

- **PSRAM 用来运行代码**

如果需要将代码放到 PSRAM 中运行,请根据实际情况修改 ld 文件,将代码的运行地址指定到 PSRAM 区间。

- **PSRAM 作为 HEAP 使用**

如果将 PSRAM 作为系统 heap 使用,此时有 2 种情况:

- **PSRAM 使用无限制:**

如果具体芯片的 PSRAM 在使用上无任何限制,对于程序来说可以等同于 SRAM。

这种情况下可以在 sram_heap 初始化之后,使用 `sysheap_add` API 将可用的 PSRAM 区间添加到 heap 管理器中。程序在使用 malloc 申请内存时就可以申请到 psram 空间的内存。同样使用 malloc/free API,程序代码可以无感知的使用 PSRAM 内存。

- **PSRAM 使用有限制:**

如果具体芯片的 PSRAM 在使用上有限制,不能完全等同于 SRAM;则需要在可以访问 PSRAM 空间的代码中使用 `os_malloc_psram/os_free_psram` API 进行申请和释放操作。这种情况下 malloc/free API 不会访问到 PSRAM 空间,而且在 system.c 中需要对 psram_heap 进行初始化,具体请查看 system.c 的 `malloc_psram_init` 函数。

- **PSRAM 作为特定 buffer 使用**

如果把 PSRAM 作为特定模块的 buffer 使用,请自行划分管理 PSRAM 区间。

5.3. 内存分配

SDK 提供了 2 种内存分配模块，分别是 `mmpool1` 和 `mmpool3`。这 2 个模块各有自己的优点：

- **mmpool1**: 分配效率更高，内存利用率更高，主要是适用小内存池和低碎片化的场景。
- **mmpool3**: 相比 `mmpool1` 分配效率和利用率都略低，更适合大内存池，可以改善内存碎片化，降低内存碎片化产生的影响。

内存池控制标识：enum `SYSHEAP_FLAGS`

```
enum SYSHEAP_FLAGS {  
    SYSHEAP_FLAGS_MEM_LEAK_TRACE      = (1u << 0), //开启内存泄漏检测  
    SYSHEAP_FLAGS_MEM_OVERFLOW_CHECK = (1u << 1), //开启内存溢出检测  
    SYSHEAP_FLAGS_MEM_ALIGN_16       = (1u << 2), //分配地址 16 字节对齐  
    SYSHEAP_FLAGS_MEM_ALIGN_32       = (1u << 3), //分配地址 32 字节对齐  
};
```

5.4. 系统 heap

SDK 默认定义了 1 个系统 heap: `sram_heap`，所有的 `malloc` 操作都从系统 heap 中分配内存。系统 heap 的大小由 `SYS_HEAP_SIZE` 宏定义控制，系统 heap 默认使用了 `mmpool1` 分配算法，如下图所示：

```

struct sys_sramheap {
    const char *name;
    const struct mmpool_ops *ops;
    struct mmpool1 pool;
};

__init void malloc_init(void)
{
    uint32 flags = 0;
#ifdef MEM_TRACE
    flags |= SYSHEAP_FLAGS_MEM_LEAK_TRACE | SYSHEAP_FLAGS_MEM_OVERFLOW_CHECK;
#endif
    sram_heap.name = "sram";
    sram_heap.ops = &mmpool1_ops;
    sysheap_init(&sram_heap, (void *)SYS_HEAP_START, SYS_HEAP_SIZE, flags);
}

```

如果要修改系统 heap 的分配模块，需要修改 2 个地方：

1. 修改 struct sys_sramheap 的定义：在 sdk/include/lib/heap/sysheap.h 文件，将 pool 字段的类型修改为 struct mmpool3。
2. 修改 heap 初始化：在 system.c 中的 malloc_init 函数，将 sram_heap.ops 修改为 mmpool3_ops。

5.5. 自定义 heap

SDK 支持添加自定义 heap 作为一些软件模块的专用内存池。SDK 提供了自定义 heap 的模板文件：xxx_heap.c 和 xxx_heap.h，基于模板文件可以简化自定义 heap 的开发工作。

添加自定义 heap 需要 2 个步骤：

1. 复制模板文件

模板文件在 sdk/lib/heap 目录下，复制 xxx_heap.c 和 xxx_heap.h 进行重命名，然后新的 .c/.h 文件中 查找替换 “xxx” 为新的内存池名称。同时确认 init 函数里面设置的 mmpool_ops 的值是否正确，默认选择使用了 mmpool1。

2. 内存池初始化

自定义 heap 模块创建后，下一步需要安排 heap 所需的内存空间，有多种方式可以选择，例如：

- 预留分配：在 `sys_config.h` 中使用宏定义为其预留空间，
- 动态分配：从其它 heap（例如系统 heap）中分配一块内存空间。

在解决所需内存空间后，在合适的地方调用 自定义 heap 的初始化，然后就可以正常使用自定义 heap 了。

```
int32 xxx_heap_init(uint32 heap_start, uint32 heap_size, uint32 flags)
```

- 参数 `heap_start`：该内存池空间的起始地址
- 参数 `heap_size`：该内存池空间的大小
- 参数 `flags`：内存池的控制标识，参见 `enum SYSHEAP\_FLAGS`。

6. SDK 开发基本规范

在使用 SDK 进行二次开发时建议遵循 SDK 定义的一些开发规范。

6.1. 头文件引用

SDK 的基础头文件都放在 sdk/include 目录下，编译工程的 include path 已添加 sdk/include 路径。部分库代码的头文件放在源码目录，需要使用时请查看具体模块代码。

SDK 提供了 `basic_include.h` 文件，该文件引用了一些 SDK 基础头文件。使用该头文件可以减少代码开发中的一些头文件引用问题。

```
#ifndef _HUGEIC_BASIC_INCLUDE_H_
#define _HUGEIC_BASIC_INCLUDE_H_

#include "typedef.h"
#include "list.h"
#include "dev.h"
#include "devid.h"
#include "errno.h"

#include "osal/atomic.h"
#include "osal/ffs.h"
#include "osal/irq.h"
#include "osal/msgqueue.h"
#include "osal/mutex.h"
#include "osal/semaphore.h"
#include "osal/sleep.h"
#include "osal/string.h"
#include "osal/task.h"
#include "osal/timer.h"
#include "osal/time.h"
#include "osal/work.h"
#include "osal/event.h"

#include "hal/gpio.h"
#include "hal/dma.h"
#include "hal/crc.h"
#include "hal/uart.h"

#include "lib/common/common.h"
#include "lib/common/sysevt.h"
#include "lib/heap/sysheap.h"
#include "lib/syscfg/syscfg.h"

#endif
```

6.2. os_xx API 说明

为了便于代码移植，SDK 对一些基础 API 进行了简单封装或宏定义封装，这类 API 的命名都是以 “os_” 开头，主要集中在 osal/string.h 文件中。

在 sdk/common/string.c 定义几个内存操作 API，例如 _os_memcpy/_os_strcpy 等。如果遇到内存溢出系统异常问题时，可以将代码中操作内存的 API 替换成 string.c 定义的这些 API，然后使用 `SYSHEAP_FLAGS_MEM_OVERFLOW_CHECK` 开启内存溢出检测功能，程序在使用这些 API 时，SDK 会对 buffer 的越界操作进行检测，对分析 buffer 溢出问题有一定的帮助。

```
#ifndef os_malloc
#define os_malloc(s) _os_malloc(s)
#endif
#ifndef os_free
#define os_free(p) do{ _os_free((void *)p); (p)=NULL;}while(0)
#endif
#ifndef os_zalloc
#define os_zalloc(s) _os_zalloc(s)
#endif
#ifndef os_realloc
#define os_realloc(p,s) _os_realloc(p,s)
#endif
#ifndef os_calloc
#define os_calloc(p,s) _os_calloc(p, s)
#endif
#ifndef os_malloc_psram
#define os_malloc_psram(s) _os_malloc_psram(s)
#endif
#ifndef os_free_psram
#define os_free_psram(p) do{ _os_free_psram((void *)p); (p)=NULL;}while(0)
#endif
#ifndef os_zalloc_psram
#define os_zalloc_psram(s) _os_zalloc_psram(s)
#endif
#ifndef os_realloc_psram
#define os_realloc_psram(p,s) _os_realloc_psram(p,s)
#endif
#ifndef os_calloc_psram
#define os_calloc_psram(p,s) _os_calloc_psram(p, s)
#endif
#endif

#define os_strcpy(d,s) strcpy((char *)(d), (const char *)(s))
#define os_strncpy(d,s,n) strncpy((char *)(d), (const char *)(s), n)
#define os_memset(s,c,n) memset((void *)(s), c, n)
#define os_memcpy(d,s,n) memcpy((void *)(d), (const void *)(s), n)
#define os_memmove(d,s,n) memmove((void *)(d), (const void *)(s), n)
#define os_sprintf(s, ...) sprintf((char *)(s), __VA_ARGS__)
#define os_vsnprintf vsnprintf
#define os_snprintf snprintf
#endif

#define os_abs abs
#define os_atoi(c) atoi((const char *)c)
#define os_atol(c) atol((const char *)c)
#define os_atoll(c) atoll((const char *)c)
```

7. SDK 设备驱动

7.1. SDK 设备管理

SDK 使用设备 ID 对设备进行统一管理, 为每个设备分配了唯一的 ID。所有的设备驱动结构体定义都必须遵循以下规则:

设备驱动结构体定义的第一个字段是必须是 struct dev_obj 类型, 或者是它的父类型 (父类型也需要遵循此规则)。

struct dev_obj 是设备的抽象层定义, 设备管理器正是基于此类型对设备进行管理。

```
struct devobj_ops{
#ifdef CONFIG_SLEEP
    int32 (*suspend)(struct dev_obj *obj);
    int32 (*resume)(struct dev_obj *obj);
#endif
};

struct dev_obj{
    uint32 dev_id;
    struct dev_obj *next;
    const struct devobj_ops *ops;
};
```

- 设备管理模块代码是 sdk/hal/dev.c。
- 设备 ID 的定义在 sdk/include/devid.h

系统初始化时需要向 SDK 注册所有需要使用的设备 (device_init 函数)。在各个设备驱动的 attach 函数中使用 dev_register API 进行注册, 在需要访问设备时使用 dev_get API 获取设备对象, 然后使用抽象层 API 访问设备。【每个设备只能注册一次】。

```
extern int32 dev_register(uint32 dev_id, struct dev_obj *device);

extern struct dev_obj *dev_get(int32 dev_id);
```

7.2. 驱动抽象层

SDK 对常见的外设都定义了抽象层。抽象层屏蔽了具体硬件差异，在应用代码开发时应使用抽象层 API，可以使应用代码很方便的在多个型号芯片的 SDK 之间进行移植。

设备驱动抽象层代码在 `sdk/hal` 目录，头文件在 `sdk/include/hal/` 目录。

`hal/dev.c` 是驱动抽象层的顶层模块，`struct dev_obj` 是所有驱动对象的父类型。

`dev.c` 设计的 API 如下：

- `int32 dev_init()`

device manger 初始化，该函数在 `system.c` 的 `pre_main` 函数中执行。

- `struct dev_obj *dev_get(int32 dev_id)`

从系统中获取指定 ID 的设备，返回类型为 `struct dev_obj *`。由于各个驱动对象均继承于 `struct dev_obj`，所以代码在使用时可以直接转换为具体的驱动类型。

- `int32 dev_register(uint32 dev_id, struct dev_obj *device)`

向系统注册设备。在各个驱动的 `attach` 函数中会向系统注册设备，设备注册后就可以使用 `dev_get` API 获取设备，访问设备。

- `int32 dev_suspend(uint16 type)`

该 API 是低功耗 API，系统在休眠前会执行 `dev_suspend` 将所有设备挂起。该功能需要设备驱动实现它的 `suspend` 函数，对设备进行挂起操作。

- `int32 dev_resume(uint16 type, uint32 wkreason)`

该 API 是低功耗 API，系统在休眠唤醒时会执行 `dev_resume` 将所有被挂起的设备恢复。该功能需要设备驱动实现它的 `resume` 函数，对设备进行恢复。

各个驱动的抽象层 API 不再一一赘述，直接阅读代码即可。

8. SDK OS 抽象层

SDK 可以支持多种 RTOS；目前已支持 AliOS 内核，鸿蒙 LiteOS 内核。为了便于代码移植和开发，SDK 对常用的 OS 功能进行抽象封装，设计了 OSAL 抽象层 API。

假设读者已具备操作系统原理基础知识，本章节仅介绍抽象层所封装的功能。

8.1. OS Task

OS Task 模块是 RTOS task 功能的抽象层，代码在 `sdk/osal/xxx/task.c`，头文件在 `sdk/include/osal/task.h`。

该抽象层对 Task 常用功能进行了封装，定义了 `struct os_task` 结构体。大部分 API 需要使用 `struct os_task` 类型。在需要创建 task 的代码中需要定义 `struct os_task` 变量。

OS Task 抽象层的 API 列表如下：

- `os_task_init`: `struct os_task` 变量初始化
- `os_task_priority`: 获取 task 的优先级
- `os_task_stacksize`: 获取 task 的堆栈大小
- `os_task_set_priority`: 设置 task 优先级
- `os_task_set_stack`: 设置 task 堆栈，堆栈大小
- `os_task_run`: 启动 task 运行
- `os_task_stop`: 停止 task 运行
- `os_task_del`: 删除 task;
- `os_task_runtime`: 获取所有 task 运行时间统计
- `os_task_current`: 获取当前运行 task 的[句柄](#)
- `os_task_data`: 根据 task [句柄](#)获取 task 关联的 data
- `os_task_suspend`: 挂起 task

- `os_task_resume`: 恢复 task 到 ready 状态
- `os_task_dump`: task 堆栈信息打印
- `os_task_yield`: 控制当前 task 让出 CPU
- `os_sched_disable`: 关调度
- `os_sched_enable`: 开调度
- `os_task_hdl2tsk`: 根据 task 句柄获取 `struct os_task` 类型数据。

在 `osal/task.h` 里面定义宏定义 `OS_TASK_INIT`, 用于简化 task 创建执行, 如下所示:

```
#define OS_TASK_INIT(name, task, func, data, prio, stack, stksize) do { \
    os_task_init((const uint8 *)name, task, (os_task_func_t)func, (uint32)data); \
    os_task_set_stack(task, stack, stksize); \
    os_task_set_priority(task, prio); \
    os_task_run(task); \
}while(0)
```

另外 `task.c` 还提供了 2 个 API: `os_task_create` 和 `os_task_destroy`, 这 2 个 API 和 `struct os_task` 结构体无关, 开发代码时也可以直接使用这 2 个 API。

8.2. OS Mutex

OS Mutex 模块是 RTOS mutex 功能的抽象封装层, 代码在 `sdk/osal/xxx/mutex.c`, 头文件在 `sdk/include/osal/mutex.h`。

该抽象层封装了 mutex 的常用功能, 定义了 `struct os_mutex` 结构体。该抽象层的 API 都基于此类型进行操作, 在需要使用 OS Mutex 功能的代码中需要定义 `struct os_mutex` 类型变量。

OS Mutex 抽象层的 API 列表如下:

- `os_mutex_init`: OS Mutex 初始化
- `os_mutex_lock`: OS Mutex 上锁操作, 该 API 需要指定 `timeout` 参数, 当 `timeout` 参数为 0 时, 就是 `try lock` 操作。
- `os_mutex_unlock`: OS Mutex 释放锁
- `os_mutex_del`: OS Mutex 删除

- **os_mutex_owner**: 获取 OS Mutex 当前被哪个 task 锁住, 发回值是 task 句柄。

8.3. OS Semaphore

OS Semaphore 模块是 RTOS semaphore 功能的抽层封装层, 代码在 sdk/osal/xxx/semaphore.c, 头文件在 sdk/include/osal/semaphore.h。

该抽象层封装了 semaphore 常用的功能, 定义 struct os_semaphore 结构体。该抽象层的 API 都基于此类型进行操作, 在需要使用 OS Semaphore 功能的代码中需要定义 struct os_semaphore 类型变量。

OS Semaphore 抽象层的 API 列表如下:

- **os_sema_init**: 信号量初始化, 可以指定信号量初始值。
- **os_sema_del**: 删除信号量
- **os_sema_down**: 对信号量执行 down 操作, 可以指定 timeout 参数, 当 timeout 参数为 0 时, 就是 [try down](#) 操作。
- **os_sema_up**: 释放信号量
- **os_sema_count**: 查看信号量的值

8.4. OS MsgQueue

OS MsgQueue 模块是 RTOS 消息队列功能的抽层封装层, 代码在 sdk/osal/xxx/msgqueue.c, 头文件在 sdk/include/osal/msgqueue.h。

该抽象层封装了消息队列常用的功能, 定义 struct os_msgqueue 结构体。该抽象层的 API 都基于此类型进行操作, 在需要使用消息队列功能的代码中需要定义 struct os_msgqueue 类型变量。

OS Msgqueue 封装的 API 仅支持往消息队列存取 **UINT32 类型**数据。

OS MsgQueue 抽象层的 API 列表如下:

- `os_msgq_init`: 消息队列初始化, 可以指定消息队列的大小。
- `os_msgq_get`: 从消息队列提取一个数据, 可以指定 `timeout` 参数, 当 `timeout` 参数为 0 时, 就是 `try get` 操作。
- `os_msgq_put`: 往消息队列存放一个数据, 可以指定 `timeout` 参数, 当 `timeout` 参数为 0 时, 就是 `try put` 操作; 消息队列可能已满, 此时 `try put` 操作不会阻塞当前 `task`。
- `os_msgq_del`: 删除消息队列
- `os_msgq_cnt`: 获取消息队列当前的消息个数

8.5. OS Timer

OS Timer 模块是 RTOS 软件 `timer` 的抽象封装层, 代码在 `sdk/osal/xxx/timer.c`, 头文件在 `sdk/include/osal/timer.h`。OS Timer 抽象层定义了 `struct os_timer` 结构体, 所有的 API 都是基于此类型进行操作。

RTOS 的软件 `timer` 是基于 OS TICK 功能实现的, 因此 OS Timer 的时间精度取决于 OS TICK 的周期。例如 当 OS HZ 为 100 (即 OS TICK 为 10ms), OS Timer 的时间精度为 10ms。

同时大多数 RTOS 的软件 `timer` 实现会使用一个 `task` 对所有的 `timer` 进行管理, 因此在 `timer` 的 `timeout` 回调函数不能执行过多的程序逻辑, 也不能有阻塞等待的行为, 此类行为会影响其他 `timer` 的触发执行, 造成不必要的 `timer delay`。

OS Timer 抽象层 API 列表如下:

- `os_timer_init`: 软件 `timer` 初始化, 可以设置 `timer` 的触发模式: 单次触发或循环触发。
- `os_timer_start`: 启动软件 `timer`, 需要指定 `timer` 周期时间, 单位为毫秒。
- `os_timer_stop`: 停止 `timer`

- `os_timer_del`: 删除 timer
- `os_timer_stat`: 查看 timer 当前是否处于活动状态。返回值 1: 活动状态, 返回值 0: 非活动状态。

8.6. OS Event

OS Event 是 RTOS 事件功能的抽象封装层, 代码在 `sdk/osal/xxx/event.c`, 头文件在 `sdk/include/osal/event.h`。OS Event 抽象层定义了 `struct os_event` 结构体, 所有 API 都是基于此类型进行操作。

RTOS 事件功能是一种实现任务间通信的机制, 主要用于实现多任务间的同步, 但事件通信只能是事件类型的通信, 无数据传输。与信号量不同的是, 它可以实现一对多, 多对多的任务间同步。即一个任务可以等待多个事件的发生: 可以是任意一个事件发生时唤醒任务; 也可以是几个事件都发生后才唤醒任务。同样也可以是多个任务同步多个事件。

OS Event 抽象层 API 列表如下:

- `os_event_init`: OS 事件对象初始化
- `os_event_del`: 删除事件对象
- `os_event_set`: 设置事件标识, 唤醒等待此事件的任务。
- `os_event_clear`: 清除事件标识
- `os_event_get`: 获取事件对象已被设置的标识
- `os_event_wait`: 等待事件标识, 可以设置等待模式, 参考 `enum OS_EVENT_WMODE`。

8.7. OS Condv

OS Condv 模块是 RTOS 条件变量功能的抽象封装层，代码在 `sdk/osal/xxx/condv.c`，头文件在 `sdk/include/osal/condv.h`。OS Condv 模块定义了 `struct os_condv` 结构体，所有 API 都是基于此类型进行操作。

OS Condv 抽象层 API 列表如下：

- `os_condv_init`: 条件变量初始化
- `os_condv_broadcast`: 唤醒所有等待的任务
- `os_condv_signal`: 至少唤醒 1 一个等待的任务
- `os_condv_del`: 删除条件变量
- `os_condv_wait`: 等待条件变量，可以设置等待超时时间，单位为**毫秒**。

8.8. OS WorkQueue/Work

OS Workqueue 是 SDK 基于 RTOS Task 功能设计的一种工作队列机制。工作队列的设计初衷是堆栈复用，节省 memory 资源。头文件是 `osal/include/osal/work.h`。

工作队列需要结合 OS Work 进行使用，将多个 OS Work 共用一个工作队列才能达到堆栈复用，节省 memory 资源的目的。OS Workqueue 会使用一个 Task 对多个 OS Work 进行调度执行。各个 OS Work 在同一个任务堆栈中执行，OS Work 完成自己的工作后立即退出，Workqueue 会继续调度执行其他 Work。因此**在 OS Work 的代码逻辑中不能有阻塞等待的行为**。

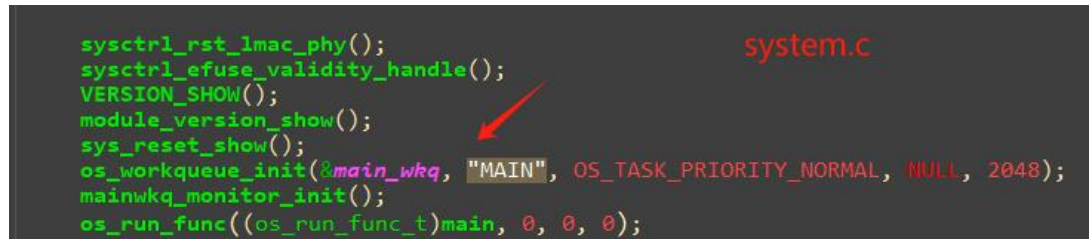
OS Work 是将一些需要反复执行，但是**优先级不高，实时性要求也不高**的代码逻辑封装起来，作为一个 Work 在工作队列执行，不需要为此类型的代码逻辑创建单独的 Task。

OS Workqueue 具有延迟执行的功能，可以方便实现一些对延时精度要求不高的定时执行功能，可以节省**软件 timer**的资源。

OS Workqueue 可以执行 OS Work，也可以执行指定的某个函数：

- **执行 OS Work:** 需要多次执行的代码逻辑, 需要创建 1 个 struct os_work 变量。
- **执行指定函数:** 仅需要执行一次的代码逻辑, 省去创建 struct os_work 变量。

SDK 在系统初始化时会创建一个**主工作队列**, 无特别需求的 work 都可以放在主工作队列中执行。



```
sysctrl_rst_lmac_phy();
sysctrl_efuse_validity_handle();
VERSION_SHOW();
module_version_show();
sys_reset_show();
os_workqueue_init(&main_wkq, "MAIN", OS_TASK_PRIORITY_NORMAL, NULL, 2048);
mainwkq_monitor_init();
os_run_func((os_run_func_t)main, 0, 0, 0);
```

The screenshot shows a code editor with the file name 'system.c' in red. The code is in C and includes several initialization functions. A red arrow points to the 'MAIN' string in the 'os_workqueue_init' function call.

main.c 文件中的 **main_wk** 就是一个 work 示例, 该 work 在主工作队列中执行, 一秒执行一次。**注意:** 在 sys_main_loop 函数添加代码, 不能有延时行为。

OS Workqueue 模块 API 列表如下:

- **os_workqueue_init:** 初始化工作队列
- **os_work_schedule:** 将 os work 加入指定的工作队列进行调度执行
- **os_work_schedule_delay:** 将 os work 加入指定的工作队列, 延迟执行
- **os_work_cancel:** 取消 os work 的执行
- **os_run_func:** 在**主工作队列**中执行指定的函数
- **os_run_func_delay:** 在**主工作队列**中延迟执行指定的函数
- **os_run_work:** 在**主工作队列**执行 os work
- **os_run_work_delay:** 在**主工作队列**延迟执行 os work

main.c 里面的 sys_main_loop 就是一个延迟循环执行的 os work 示例。

宏定义 **OS_WORK_INIT** 用于对 os work 进行初始化。

9. SDK 事件消息

SDK 设计了事件消息模块，采取**订阅-发布**的交互机制。各个软件模块在运行过程中可以产生并发布相应的事件，事件发布后系统会通知订阅此事件的模块，通知其采取相应的处理。

事件的**发布者**和**订阅者**彼此之间没有直接联系。发布者不需要关心是谁需要处理此事件，而订阅者不需要关心是谁产生了此事件，只需要关注如何处理此事件。发布一个事件可以被多个订阅者接收处理。

9.1. 事件类型

SDK 对事件消息进行了统一的管理，对事件进行了分类编号。每个事件都有一个唯一的 32 位 ID 编号。这个 32 位的事件 ID 包含了 16 位主 ID 和 16 位子 ID。

宏定义 SYS_EVENT 用来定义事件 ID:

```
#define SYS_EVENT(main, sub) ((main)<<16|(sub&0xffff))
```

在 sdk/include/lib/common/sysevt.h 文件中可以看到 SDK 已经支持的事件类型和 ID。枚举 SYSEVT_MAINID 定义了事件主 ID，表示某一类事件；然后每一类事件再定义自己的事件子 ID，例如：

```
enum SYSEVT_MAINID { /* uint16 */
    SYS_EVENT_NETWORK = 1, //mainID: for network events
    SYS_EVENT_WIFI,      //mainID: for wifi module
    SYS_EVENT_LMAC,       //mainID: for lmac module
    SYS_EVENT_SYSTEM,     //mainID: for system
    SYS_EVENT_BLE,        //mainID: for BLE
};

enum SYSEVT_SYSTEM_SUBEVT { /* uint16 */ //subID for system events.
    SYSEVT_SYSTEM_RESUME = 1, //subID: system resumed
    SYSEVT_SYSTEM_SD_MOUNT,   //subID: SD card mounted
    SYSEVT_SYSTEM_SD_UNMOUNT, //subID: SD card unmount
};

enum SYSEVT_WIFI_SUBEVT { /* uint16 */ //subID for wifi events.
    SYSEVT_WIFI_CONNECT_START = 1, //subID: wifi start connect
    SYSEVT_WIFI_CONNECTED,        //subID: wifi connected
};
```

```

SYSEVT_WIFI_CONNECT_FAIL,           //subID: wifi connect fail
SYSEVT_WIFI_DISCONNECT,             //subID: wifi disconnect
SYSEVT_WIFI_SCAN_START,             //subID: wifi start scan
SYSEVT_WIFI_SCAN_DONE,              //subID: wifi scan complete
SYSEVT_WIFI_STA_DISCONNECT,          //subID: sta disconnect - for AP
SYSEVT_WIFI_STA_CONNECTED,          //subID: sta connected - for AP
SYSEVT_WIFI_STA_PS_START,           //subID: sta enter sleep mode
SYSEVT_WIFI_STA_PS_END,             //subID: sta exit sleep mode
SYSEVT_WIFI_PAIR_DONE,              //subID: pairing DONE
SYSEVT_WIFI_TX_SUCCESS,              //subID: data tx success
SYSEVT_WIFI_TX_FAIL,                //subID: data tx fail
SYSEVT_WIFI_UNPAIR,                 //subID: unpaired
SYSEVT_WIFI_WRONG_KEY,              //subID: wrong password
};

```

9.2. 发布事件

使用 `sys_event_new` API 发布一个事件，任意模块的代码都可以发布自己的事件。

发布事件时可以携带一个 event data，类型是 `uint32`，含义自定义。只有在处理事件的代码才需要知道 event data 的含义。

发布事件的 API:

```
int32 sys_event_new(uint32 event_id, uint32 data);
```

参数:

- **event_id:** 事件 ID (mainID+subID)
- **data:** 每个事件消息，可以携带一个 `uint32` 自定义数据。

为了方便进行事件发布操作，每一类事件都定义了一个自己的宏定义，例如:

```

#define SYSEVT_NEW_SYSTEM_EVT(subevt, data) // 发布 system 事件消息
#define SYSEVT_NEW_LMAC_EVT(subevt, data)   // 发布 LMAC 类型的事件消息
#define SYSEVT_NEW_WIFI_EVT(subevt, data)   // 发布 WiFi 类型的事件消息
#define SYSEVT_NEW_NETWORK_EVT(subevt, data) // 发布 network 类型的事件消息
#define SYSEVT_NEW_BLE_EVT(subevt, data)    // 发布 BLE 类型的事件消息

```

开发者可以继续添加事件定义，为自己的代码添加 mainID 和 subID。

9.3. 订阅事件

事件消息被发布后，由 SDK 进行统一管理。SDK 会查询此事件的订阅信息，执行订阅者的回调函数。

订阅事件的 API:

```
int32 sys_event_take(uint32 evt_id, sysevt_hdl hdl, uint32 priv)
```

参数:

- **evt_id** : 订阅的事件 ID。evt_id 的值可以是 **0xffffffff**, 主 ID(子 ID 为 0), 或者具体事件 ID (主 ID+子 ID)
 - **0xffffffff**: 表示订阅所有的事件
 - **主 ID**: 表示订阅某一类事件, 例如: **SYS_EVENT(SYS_EVENT_WIFI, 0)**, 表示所有的 WiFi 事件 (子 ID 填 0)。
 - **事件 ID**: 只订阅某个具体的事件。例如: **SYS_EVENT(SYS_EVENT_WIFI, SYSEVT_WIFI_CONNECTED)**, 只订阅 WiFi 连接事件。
- **hdl** : 订阅事件的回调函数。该回调函数的返回值定义:
 - **SYSEVT_CONTINUE**: 该事件可以继续传递给其他订阅者;
 - **SYSEVT_CONSUMED**: 该事件已被回调函数消耗, 不能继续传递给其他订阅者。
- **priv** : 自定义私有数据, 在执行回调函数时 priv 参数会传递给回调函数。

事件回调函数定义:

```
typedef sysevt_hdl_res(*sysevt_hdl)(uint32 event_id, uint32 data, uint32 priv);
```

回调函数的参数:

- **event_id**: 发布的事件 ID
- **data**: 该事件的数据 (每个事件的数据含义会不一样)
- **priv**: 订阅者的私有数据 (也就是 sys_event_take 函数的第 3 个参数)

10. SDK POSIX 接口

为了支持移植第三方代码, SDK 实现了部分 POSIX 接口。代码在 `sdk/lib/posix` 目录, 头文件在 `sdk/include/lib/posix` 目录。实现的功能包括 `pthread` API, `socket` API 和 `file system` API。

使用 POSIX 接口的代码需要添加头文件: `sdk/include/lib/posix/stdio.h` 和 `sdk/include/lib/posix/pthread.h`。

10.1. FS 接口

在 `sdk/lib/posix/stdio.c` 定义常用的 `fs api`, 按常规使用方法使用即可。

10.2. Socket 接口

为了兼容移植的第三方代码使用 `read/write` API 访问 `socket`。在 `sdk/lib/posix/stdio.c` 中实现了 `read/write` 函数, 该函数根据 `fd` 的大小识别是执行 `socket` API, 还是 `FS` API。其他的 `socket` API 请使用 `lwip` 的 `socket` API。

10.3. Pthread 接口

SDK 实现了 `pthread` API, 包括 `pthread`, `pthread` 互斥锁, `pthread` 条件变量, `pthread` 读写锁, `pthread` 消息队列。参考 POSIX `pthread` 接口规范使用即可。

11. SDK 参数管理

SDK 的系统参数管理默认是双备份, 在 Flash 划分了 2 个参数区。参数保存和加载时, SDK 自动处理参数区的切换。如果在参数保存过程中意外断电或复位, 系统开机后会识别到损坏的参数区, 加载上一次保存的参数区。

11.1. API 列表

SDK 参数管理模块的 API 在 `sdk/lib/syscfg/syscfg.h`。

enum SYSCFG_ERASE_MODE 定义了参数保存时 flash 的擦除模式

```
enum SYSCFG_ERASE_MODE {  
    SYSCFG_ERASE_MODE_SECTOR, //按 sector 擦除  
    SYSCFG_ERASE_MODE_BLOCK,  //按 block 擦除  
    SYSCFG_ERASE_MODE_CHIP,    //整个 flash 擦除 (谨慎! 危险!)  
};
```

struct syscfg_info 定义了参数区的分区信息, 指定每个参数分区所在的 flash 和地址, 参数区的大小, 以及擦除模式。

```
struct syscfg_info {  
    struct spi_nor_flash *flash1, *flash2; //参数区 1, 参数区 2 所在的 flash  
    uint32 addr1, addr2; //参数区 1, 参数区 2 的地址  
    uint32 size; //参数区大小  
    uint8 erase_mode; //擦除模式  
};
```

API 列表如下:

- `int32 syscfg_init(const char *name, void *cfg, uint32 size);`
参数管理功能初始化, 仅在系统初始化执行一次, 同时会从 flash 加载参数。
参数说明:
 - `name`: 参数区名称, 根据名称确定参数区的存储位置。
 - `cfg`: 加载参数的变量地址
 - `size`: 加载参数的大小
- `int32 syscfg_read(const char *name, void *cfg, uint32 size);`
从 flash 读取参数。
参数说明:
 - `name`: 参数区名称, 根据名称确定参数区的存储位置。
 - `cfg`: 加载参数的变量地址
 - `size`: 加载参数的大小
- `int32 syscfg_write(const char *name, void *cfg, uint32 size);`
保存参数到 flash。

参数说明:

- name: 参数区名称, 根据名称确定参数区的存储位置。
- cfg: 参数变量地址
- size: 参数大小

- int32 `syscfg_info_get`(struct syscfg_info *pinfo, const char *name);
获取参数区分区信息, 该函数在 device.c 中实现。

参数说明:

- pinfo: 用于填充参数区的存放位置信息
- name: 参数区的名称, 根据名称确定参数区的存储位置。在实现 `syscfg_info_get` 时根据 name 识别各个参数功能, 分配对应的存储位置。
 - ◆ 系统参数区名称为 "syscfg", 旧版本 SDK API 没有这个参数, 只支持一个系统参数。

- void `syscfg_loaddef`(const char *name);
参数区恢复出厂默认设置。

11.2. 参数分区

SDK 的参数支持双备份存放, 请根据实际方案需要指定各个参数区的存放位置。SDK 默认将系统参数存放在 flash 末尾的 2 个 sector 位置, 参数区大小为 4K byte。

device.c 中的 `syscfg_info_get` 函数指定了参数区的存放信息, 如下图所示:

```
int32 syscfg_info_get(struct syscfg_info *pinfo, const char *name)
{
    if(os_strcmp(name, "syscfg") == 0){
        pinfo->flash1 = &flash0;
        pinfo->flash2 = &flash0;
        pinfo->size = pinfo->flash1->sector_size;
        pinfo->addr1 = pinfo->flash1->size - (2 * pinfo->size);
        pinfo->addr2 = pinfo->flash1->size - pinfo->size;
        printf("pinfo->size:%d sys_config:%d sector_size:%d\r\n", pinfo->size,
            sizeof(struct sys_config), pinfo->flash1->sector_size);
        ASSERT((pinfo->addr1 & ~(pinfo->flash1->sector_size - 1)) == pinfo->addr1);
        ASSERT((pinfo->addr2 & ~(pinfo->flash2->sector_size - 1)) == pinfo->addr2);
        ASSERT((pinfo->size >= sizeof(struct sys_config)) &&
            (pinfo->size == (pinfo->size & ~(pinfo->flash1->sector_size - 1))));
    }
    return 0;
}
```

11.3. 参数扩展

`struct sys_config` 结构体是 SDK 的系统参数区定义。开发者可以将自己的参数添加到系统参数中，但是只能在 **该结构体末尾新增字段**。当扩展后的 `sizeof(struct sys_config)` 超过参数区大小时（默认 4K），就需要调整参数区的划分（`syscfg_info_get` 函数）。

11.4. 自定义参数区

SDK 默认只有 1 个参数区：`syscfg`，如果有某些功能的参数不希望和系统参数一起存放，则可以自定义参数区。自定义参数区时需要规划参数区的存放位置，定义参数区的名称，在 `syscfg_info_get` API 中根据 name 识别需要访问的参数区，返回对应的存储位置信息。在使用 `syscfg_read/syscfg_write` API 时传入自定义参数区的名称即可。

12. SDK C++代码支持

SDK 支持添加 C++代码。添加 C++代码后在系统初始化时需要在 main 函数中执行 `do_global_ctors` 函数，该函数是对 C++的全局变量进行初始化。

同时需要检查 ld 文件是否已经添加.ctors 和.dtors 的处理。

例如：

```
. = ALIGN(0x4) ;
__CTOR_LIST__ = . ;
LONG((__CTOR_END__ - __CTOR_LIST__) / 4 - 2)
KEEP(*(SORT(.ctors)))
LONG(0)
__CTOR_END__ = . ;
__DTOR_LIST__ = . ;
LONG((__DTOR_END__ - __DTOR_LIST__) / 4 - 2)
KEEP(*(SORT(.dtors)))
LONG(0)
__DTOR_END__ = . ;
. = ALIGN(0x4) ;
```

13. SDK 内置小模块

SDK 内置一些小模块，实现了一些常用的功能函数。开发代码时可以直接使用这些小模块，节省一点开发工作。

13.1. ringbuffer

ringbuffer 模块实现了循环 buffer 的功能，用于单 reader，单 writer 的场景，可以免锁访问，减少系统开销。

代码在 sdk/lib/common/rbuffer.c，头文件在 sdk/include/common/rbuffer.h。

根据 ringbuffer 存放数据元素的长度特征，有 2 种使用方式：

- **数据大小固定**：在定义 ringbuffer 时可以指定数据元素类型，可以是基础类型，也可以是自定义结构体类型。使用 RB_GET/RB_SET 可以一次操作一个元素。
- **数据大小变长**：如果存放的数据大小是可变的，定义 ringbuffer 时只能指定 char 类型。使用 rbuffer_set/rbuffer_get API 对数据进行操作。

ringbuffer 设计了 2 种 API：

- **宏定义 API**：使用宏定义 API，可以将 ringbuffer 内嵌到其他模块的定义中，适用于数据元素大小固定的场景。
 - `#define RBUFFER_DEF(name, type, size)` //定义 ringbuffer，指定元素类型和 buffer 大小（元素的个数），以数组形式自动分配 buffer 空间。需要配合 `RB_INIT` 宏定义进行初始化。
 - `#define RBUFFER_DEF_R(name, type)` //定义 ringbuffer，仅指定元素类型，buffer 空间由外部分配，需要配合 `RB_INIT_R` 宏定义进行初始化。
 - `#define RB_GET(rb, val)` //从 ringbuffer 获取一个元素
 - `#define RB_SET(rb, val)` //向 ringbuffer 存入一个元素

- `#define RB_INIT(rb, size)`//ringbuffer 初始化, 配合 `RBUFFER_DEF` 使用

- `#define RB_INIT_R(rb, size, buff)`//ringbuffer 初始化, 需要指定 buffer 空间 (由外部分配和释放), `RBUFFER_DEF_R` 使用。

- `#define RB_INT_GET(rb, val)`//关中断后从 ringbuffer 获取一个元素

- `#define RB_INT_SET(rb, val)`//关中断后向 ringbuffer 存入一个元素

`RB_INT_GET` 和 `RB_INT_SET` 这 2 个 API 建议在符合以下条件的情况下使用:

1. ringbuffer 的存储数据是基础类型, 例如 `uint8`, `uint32`, 指针, ...
2. ringbuffer 存在多个 reader 或多个 writer 同时访问的情况, 或者需要中断上下文执行 get/set 操作。通过关中断进行 get/set 操作, 既保证互斥操作, 又减少了上锁的开销。

- 函数 API: 使用函数 API 对 ringbuffer 进行访问, 适用于数据元素大小可变的场景。使用 `struct rbuffer` 对 ringbuffer 进行定义。

- `int32 rbuffer_init(struct rbuffer *rb, uint32 size, void *buff);`
ringbuffer 初始化, 指定 buffer 大小和 buffer 空间地址。

- `int32 rbuffer_set(struct rbuffer *rb, void *data, uint32 length);`
向 ringbuffer 写入数据。

- `int32 rbuffer_set_force(struct rbuffer *rb, void *data, uint32 length);`
向 ringbuffer 写入数据, 当 ringbuffer 已满时, 自动覆盖旧的数据。

- `int32 rbuffer_get(struct rbuffer *rb, void *buff, uint32 size);`
从 ringbuffer 中读取数据。

- `void rbuffer_destroy(struct rbuffer *rb);`
销毁 ringbuffer。

- `void rbuffer_reset(struct rbuffer *rb);`
ringbuffer 复位, 所保存的数据丢失。

13.2. 字符串转换

SDK 内置了一些字符串处理函数，代码在 `sdk/lib/common/string.c`。

- **hexchr2int** : hex 字符 转换成 int 数字，例如字符 ‘A’ 转换成数字 0x0A。
- **hex2char**: hex 字符串 转换成 char 数字，例如字符串 “0xAA” 转换成数字 0xAA。
- **hex2bin**: hex 字符串 转换成 char 数组，例如字符串 “1122334455” 转换成数组 {0x11, 0x22, 0x33, 0x44, 0x55}。
- **str2mac**: mac 地址字符串 转换成 mac 地址
- **str2ip**: ip 地址字符串 转换成 IP 地址
- **os_atoh**: hex 字符串 转换成 32 位数字
- **os_atohl**: hex 字符串 转换成 64 位数字
- **os_strdup**: 字符串拷贝函数

13.3. print 输出

SDK 重新设计了 print 输出函数，支持输出重定向和打印级别，以及 console 颜色控制。代码在 `sdk/lib/common/string.c`。

- 打印输出缓冲区

打印缓冲区大小默认是 256byte，由宏定义 `PRINT_BUFF_SIZE` 控制，可以在 `project_config.h` 中重定义 `PRINT_BUFF_SIZE` 来修改缓冲区大小。

- **hgprintf**

SDK 实现的打印输出函数，库函数 `printf` 也会被 `alias` 重定向到 `hgprintf` 函数。

- **os_printf**

带[时间戳](#)的打印输出，该宏定义调用了 `hgprintf`，添加了时间戳控制信息。

- **_os_printf**

不带时间戳的打印输出，等效于 `hgprintf`。

- 打印输出等级

SDK 支持打印等级控制和 console 颜色控制。默认的打印等级是 0，允许所有的打印信息输出。`print_level` 函数可以修改打印等级，大于设定的打印等级的信息将会被过滤不输出。SDK 定义了 8 种打印等级，不同等级的 console 输出颜色也会不同。

```
#define KERN_EMERG      KERN_SOH"0" /* system is unusable */
#define KERN_ALERT      KERN_SOH"1" /* action must be taken immediately */
#define KERN_CRIT       KERN_SOH"2" /* critical conditions */
#define KERN_ERR        KERN_SOH"3" /* error conditions */
#define KERN_WARNING    KERN_SOH"4" /* warning conditions */
#define KERN_NOTICE     KERN_SOH"5" /* normal but significant condition */
#define KERN_INFO       KERN_SOH"6" /* informational */
#define KERN_DEBUG      KERN_SOH"7" /* debug-level messages */
```

- 打印输出重定向

SDK 支持对打印信息输出进行重定向。利用此特性可以将调试信息输出到主控、网口或其他任意输出界面。可实现类似 Telnet，SSH 的操作界面。

使用 `print_redirect` API 进行打印重定向。该 API 执行后，系统的所有打印信息都将输出到指定的输出函数。

```
void print_redirect(osprint_hook print, void *priv)
```

- `dump_hex`

将 buffer 中的数据以 hex 字符格式打印输出。

- `dump_key`

将 buffer 中的数据以 hex 密码格式输出。

- `os_memdup`

内存数据拷贝函数。

- `os_random_bytes`

伪随机数产生函数

- `void print_level(int8 level)`

设置 SDK 打印输出级别，低于设置的 level 的日志才会被输出

- `void disable_print(int8_t dis)`

关闭 SDK 打印输出

- `void disable_print_clore(int8_t dis)`

关闭 SDK 打印输出的颜色控制

13.4. errlog 功能

在产品测试、运行过程中有可能会出现问题，但是又不方便抓取异常日志。为了能抓取异常日志进行有效分析，SDK 设计了 errlog 模块。

errlog 模块的功能是将低于指定 level 的日志信息保存到 flash 中。当设备出现异常时该模块会将出现异常时的日志信息保存下来，可以在设备重启后再查看异常信息。

errlog API 列表:

`void sys_errlog_init(int8 level, uint16 buf_size);`

errlog 功能默认是关闭状态，需要使用 sys_errlog_init API 打开。

- **参数 level:** 需要保存的日志级别，小于等于 level 的日志信息会被保存下来
- **参数 buf_size:** errlog 功能的 buffer size。errlog 功能需要使用 2 个 buffer 减小日志缓存，避免频繁擦写 flash。buffer size 通常设置为 flash 的 sector size 或者 sector size/2。

`void sys_errlog_flush(uint32 p1, uint32 p2, uint32 p3);`

sys_errlog_flush API 用于通知 errlog 模块将缓存的异常日志刷写到 flash。默认情况下 errlog 模块在缓存的日志将填满 buffer 时自动写入 flash，但是程序可以在检测异常时主动要求 errlog 刷写日志。

- 参数 p1 : 0xffffffff: 表示强行刷写日志。
- 参数 p2 : 未使用

- 参数 p3 : 未使用

```
void sys_errlog_dump(void);
```

sys_errlog_dump API 用于打印 errlog 模块保存的日志信息。该 API 会将 flash 里面保存的日志信息全部打印出来。

使用 errlog 功能还需要指定 errlog 在 flash 中的存放位置,使用 errlog_flashinfo 函数指定 errlog 的存储区域,如下图所示:

```
int32 errlog_flashinfo(struct spi_nor_flash **flash, uint32 *addr, uint32 *size)
{
    //errlog 存放在flash末尾, syscfg之前的4个sector
    *flash = &flash0;
    *size = 4*flash0.sector_size;
    *addr = flash0.size - 2*flash0.sector_size - *size;
    return RET_OK;
}
```

errlog_flashinfo 函数在 device.c 文件中,如果该文件没有这个函数,可以自行添加。根据设备的 flash 分区情况,为 errlog 指定一块区间保存 errlog 信息。

指定的地址和大小,应该符合 flash 的擦写特性,例如 addr 是 sector size 的整数倍位置, size 也是 sector size 的整数倍。errlog 模块会根据 size 的大小计算有几个 sector 区间,循环擦写这一片区域。

14. SDK 固件生成

14. 1. 固件生成过程

14. 2. 固件烧录

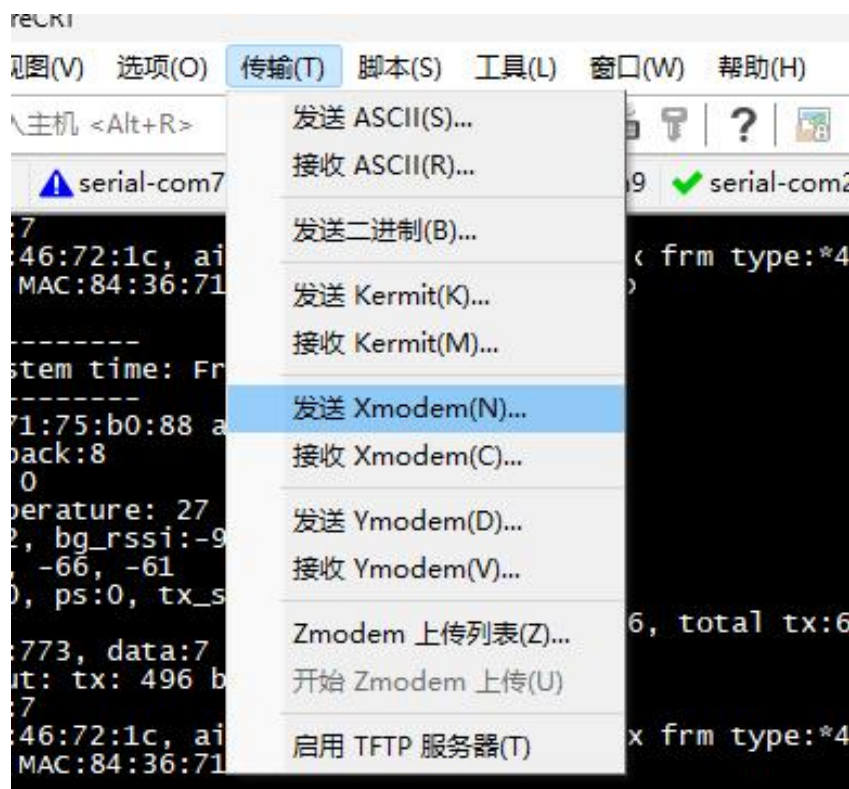
15. SDK 调试辅助

15.1. 串口固件升级

SDK 默认支持串口 Xmodem 协议升级固件，该功能通常是在开发调试阶段使用。

在串口输入 `at+fwupg` 指令，芯片进入 OTA 模式。串口连续输出 CCCCCC，表示芯片已进入升级模式。然后通过串口工具使用 Xmodem 协议发送固件文件。

注：secureCRT 软件支持 Xmodem 协议，如图所示：



15.2. CPU 使用率

SDK 支持打印输出 CPU 使用率信息。`cpu_loading_print()` 函数可以打印 CPU 使用率，该打印信息显示了各个 Task 的 CPU 使用情况。从该信息可以分析代码运行是否存在异常。

SDK 默认支持 “`at+sysdbg=top,1`” 命令，该命令可以开启周期性打印 CPU 使用率信息。默认的打印周期为 5 秒。

```
-----
Task Runtime Statistic, interval:6013ms
PID   Name          %CPU(Time)   Stack  Prio          Status
-----
1     idle_task      98%( 5893)   256    61 (20002c6c) Ready
2     timer_task     0%( 3)       512    5 (20003254) Pend
3     MAIN           0%( 16)      2048   43 (2000711c) Ready
5     lmac tx status 0%( 2)       1024   26 (20009b70) Pend
7     lmac rx        0%( 42)      1024   25 (2000a764) Pend
8     lmac main      0%( 8)       1024   43 (2000acfc) Pend
9     hw             0%( 42)      3072   26 (2000bb48) Pend
-----
```

如果因为系统 Task 异常造成周期性打印无法正常输出时，可以执行 `at+top` 指令查看 CPU 使用率，该指令执行一次仅打印一次。`at+top` 指令是在 uart 中断中执行，只要系统未死机，该指令就可以正常输出。

中断统计

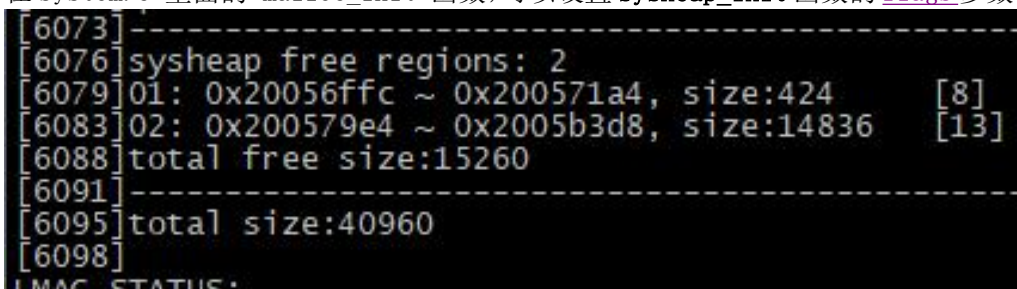
在分析 CPU 使用率时，也可以把中断执行时间打印出来。在 `project_config.h` 文件添加宏定义 `SYS_IRQ_STAT`，可以打开 SDK 的中断统计功能，统计各个中断的触发次数，单次最大执行时间和统计周期内总的执行时间。用于分析中断服务程序是否存在异常。打开此功能后，在周期性打印 CPU 使用率时也会同时打印中断的统计信息。

15.3. 内存分配信息

SDK 支持打印输出 SDK 内存分配情况，可以查看 heap 总大小，剩余大小。支持内存分配追踪，有助于分析内存使用情况，以及内存泄漏问题。

SDK 默认支持 “at+sysdbg=heap,1” 命令，执行该命令可以开启周期性打印内存分配情况，打印周期默认为 5 秒。

在 system.c 里面的 malloc_init 函数,可以设置 sysheap_init 函数的 [flags](#) 参数。



```
[6073]-----
[6076]sysheap free regions: 2
[6079]01: 0x20056ffc ~ 0x200571a4, size:424      [8]
[6083]02: 0x200579e4 ~ 0x2005b3d8, size:14836    [13]
[6088]total free size:15260
[6091]-----
[6095]total size:40960
[6098]
```

上述图片展示了内存剩余大小（共 2 个内存片段）和总大小。

使用 SDK 定义的 os_memcpy/os_strcpy 等内存 copy 函数，可以借助上述的内存越界检查功能，进行内存越界定位，代码实现在 sdk/lib/common/string.c。

自定义 heap 的调试，可以在自定义 heap 的 init 函数指定 flags 参数，然后使用自定义 heap 的 status 函数，打印 heap 的分配情况。

15.4. Task 状态分析

如果系统运行过程中出现 Task 卡死/阻塞的情况，可以借助 SDK 的 top 指令，导出 Task 堆栈信息进行分析。at+top 指令可以打印当前系统 Task 的状态信息，如下所示：

```
[70974283]-----
[70974283]Task Runtime Statistic, interval:951ms
[70974283]PID      Name                %CPU(Time)    Stack  Prio      Status
[70974283]-----
[70974283]0        dyn_mem             0%( 0)        256  60 (20002f70) Pend
[70974283]1        idle_task          99%( 945)     256  61 (20003300) Ready
[70974283]2        timer_task         0%( 0)        512  5 (20003240) Pend
[70974283]3        usbh               0%( 0)        4096 51 (20007b68) Pend
[70974283]4        MAIN              0%( 0)        2048 43 (200084f8) Pend
[70974283]5        lmac tx            0%( 0)        1024 27 (20009990) Pend
[70974283]6        lmac tx status     0%( 0)        1024 26 (20009e10) Pend
[70974283]7        lmac beacon task   0%( 0)        1024 19 (2000a290) Pend
[70974283]8        lmac rx            0%( 2)        1024 25 (2000aa04) Pend
[70974283]9        lmac main          0%( 0)        1024 43 (2000af94) Pend
[70974283]10       hw                0%( 3)        2048 26 (2000b9b8) Pend
[70974283]11       hw_rx            0%( 0)        1024 27 (2000be60) Pend
[70974283]12       fwtsk            0%( 0)        1024 28 (2000ce28) Pend
[70974283]13       tcpip_thread      0%( 0)        1024 35 (2000df14) Pend
[70974283]14       skmntr            0%( 1)        768  43 (2000e3a0) Pend
[70974283]15       GMAC rx           0%( 0)        512  20 (20011234) Pend
[70974283]-----
```

上图中优先级后面括号里的值是 Task 句柄。

执行 at+dumptask 指令可以打印 Task 的堆栈信息。该指令需要指定 Task 句柄，在 top 指令输出信息里面可以找到，如上图箭头所示。

```
Task:hw_rx
task_state: 2
stack_size: 1024
task_stack: 0x2000ba60,0x2000bdc4,0x2000be60
stack dump:
0x80000140,0x00000001,0x2000be60,0x200084f8,
0x80000140,0x2000ba38,0xffffffff,0xffffffff,
0x80000140,0x20003660,0x10101010,0x11111111,
0x00000000,0x0000001b,0x18012164,0x80000140,
0x18000a62,0x28282828,0x200035a8,0x1801253e,
0x2000ba38,0xffffffff,0x06060606,0x07070707,
0x20003728,0x180110a8,0x18010712,0x04040404,
0x2000364c,0x1802a32a,0x04040404,0x05050505,
0x06060606,0x07070707,0x00000000,0x09090909,
0x10101010,0x18010794,0x18012e08,
[70974283]-----
```

at+dumptask=0x2000be60

task stack data

导出 Task 的堆栈信息后，可以结合 asm 文件进行分析，追溯还原 Task 的 callstack 信息。

在 sdk/tools 目录下提供了 alios_stack.exe 工具，可以解析还原部分芯片的 Task 堆栈数据（目前只支持 CK803/CK802 CPU 的芯片）。使用方法：

◆ 将 at+dumptask 命令的输出信息保存为 xx_task.txt，保存到 sdk/tools 目录下。

◆ 执行 .\alios_stack.exe ck803 ..\project\Lst\xxxx.asm xx_task.txt

该工具正确解析后，会打印出该 Task 的 callstack 信息，借助此信息进一步分析 Task 的异常情况。